

Agroverse Chocolate Subscriptions — pre-flight + execution roadmap

For: TrueSight DAO / Agroverse contributors
From: Claude (agentic AI) — design ratified with Gary Teh
Date: 2026-06-09
Status: Roadmap aligned · implementation begins at Phase 1 PR1.1

Goal: Let supporters subscribe to a recurring monthly shipment of Agroverse ceremonial-cacao **chocolate bars** (generic, vintage-independent), pick the quantity themselves, manage/cancel without operator involvement, and have fulfillment land with **near-zero manual relay** between Gary and Kirsten. First real subscriber: **Linda (Rochester, NY)**. The self-serve subscribe page is also intended as a **cacao-circle event placard QR** target.

► RESUME HERE

► **ACTIVE: Phase 1, PR1.1 — generic SKU + subscription schema** in `products.js` (`agroverse_shop_beta`). Then PR1.2 (shared subscribe engine) → PR1.3 (the `/subscribe/chocolate-bar/` clean-URL wrapper) → PR1.4 (GAS action) → PR1.5 (generic-bar PDP). Architecture is **one data-driven engine, clean path URLs, catalog-as-source-of-truth** — see *Decisions + Generic SKU model + PDP spec*. Nothing implemented yet (doc written 2026-06-09, architecture folded in 2026-06-09). Start at the **Pre-flight checklist** (GAS deploy path + generic SKU definition), then PR1.1. Each PR is independently shippable; after any PR merges, report the DAO contribution before starting the next (see `DAO_CLIENT_AI_AGENT_CONTRIBUTIONS.md`).

Companion docs (read before touching the relevant layer): - `STRIPE_LEDGER_ROUTING.md` — all 5 Stripe flows; the `[LEDGER_ID]` routing pattern; the Stripe Social Media Checkout ID audit tab. - `notes/claude_serialized_qr_sales_2026-04-29.md` — the one-`[SALES_EVENT]`-per-QR-code pattern + fee amortization (the fulfillment fan-out reuses this verbatim). - `EDGAR_DAO_EXTRACTION_PLAN.md` — Edgar→`dao_protocol` split. **The Stripe webhook ENTRY (`/stripe_webhook`) intentionally stays on Rails (`sentiment_importer`)**; new DAO-glue read endpoints go to `dao_protocol (:8010)`. - `DAPP_PAGE_CONVENTIONS.md` + `dapp/UX_CONVENTIONS.md` — for the dapp fulfillment page (Phase 2) and the `agroverse_shop` sign-in (Phase 3).

Design summary (the one insight)

The current one-off flow already records the **physical reality after the fact**: Kirsten picks whatever bars are in stock, then the actual QR codes that shipped are recorded via `report_sales.html`. The clumsiness is the human *relay*, not the model. So:

- **Never pre-assign QR codes.** A subscription is a *recurring charge + a monthly fulfillment obligation*. The specific QR codes are bound at **pack time** by Kirsten, in one batch action. This dissolves the “what if Kirsten can’t find that exact bar” problem — she grabs any N in-stock bars and records what she grabbed.

- **Generic, vintage-independent SKU.** Decouples the subscription commitment from any single harvest. **Traceability is preserved at the unit level** — every bar still carries its QR → vintage → farm; the subscriber *discovers* provenance by scanning (the Cacao Chasers surprise model), they just don't pre-select.
- **Fulfillment = N existing [SALES EVENT] s**, not a new event type. The entire QR→ledger→treasury chain runs unchanged; revenue auto-routes to each bar's own AGL ledger via its `ledger_shortcut`. Only genuinely new pieces: the obligation **queue** and a **batch-submit UI** that loops (≈ `dao_client/examples/bulk_qr_sales_template.py`).
- **Identity = verified email, server is source of truth.** Reuse the canonical RSA flow (capoeira/oracle/dapp). Cross-browser portability is solved by making the verified email the key and reading orders/subscriptions **server-side** — `localStorage (agroverse_order_history)` becomes a cache, never synced between browsers. Subscription management = a **Stripe Customer Portal** link minted for the matching Customer (works from any browser).

Decisions (ratified with Gary, 2026-06-09)

#	Decision
Scope	Chocolate bars only for now (no generic ceremonial-cacao SKU yet).
Billing engine	Stays in the existing GAS (<code>agroverse_shop_checkout.gs</code> → <code>createCheckoutSession</code>). Subscriptions reuse the same dynamic inline <code>price_data</code> , adding <code>recurring:{interval:'month'} + mode:'subscription'</code> . No pre-created Stripe Price IDs. Confirmed: Stripe Checkout supports on-the-fly recurring <code>price_data</code> .
Quantity	User-chosen bars/month on the subscribe page (also the event-placard QR target).
Pricing	Same basis as the shop today — generic bar unit price × quantity (dynamic), computed server-side.
Shipping	Computed from signup address — run EasyPost once at signup, lock that amount as a recurring shipping line item (subscription mode has no interactive <code>shipping_options</code> picker, so a recurring line is the correct mechanism).
Fulfillment attribution	On each per-bar <code>[SALES EVENT]</code> : Sold by = Kirsten Ritschel, Cash proceeds collected by = Gary Teh (same as today).
Start order	Phase 1 (the subscribe page) first ; Linda receives that page's prod URL (no throwaway zero-code Stripe link).
Identity	Adopt the canonical RSA email-bound flow on <code>agroverse_shop</code> (Phase 3); server-side read-by-email retires the <code>localStorage</code> -only history limitation.
Beta-first	All UI work lands in <code>agroverse_shop_beta</code> / <code>dapp_beta</code> ; human promotes to prod. Never push prod directly.
Page architecture	One data-driven subscribe ENGINE, not a page per product. A single <code>/subscribe/</code> engine renders entirely from the catalog entry; each subscribable SKU is exposed via a thin clean-URL wrapper — <code>/subscribe/chocolate-bar/</code> now, <code>/subscribe/ceremonial-cacao/</code> later. Adding a new generic line = a catalog entry + a ~10-line wrapper, no new engine, GAS action, or fulfillment code . Clean path URLs (not <code>?sku=</code>) for SEO / sharing / placard QR.
Generic SKU model	Each generic, vintage-independent line = one catalog entry per GTIN in <code>products.js</code> , carrying subscription metadata (<code>subscribable</code> , <code>cadence</code> , <code>min/max/defaultQty</code>). The SAME entry feeds two render surfaces : the existing PDP system (one-off buy + Subscribe CTA) and the shared subscribe engine . Origin is <code>rotating</code> (not one farm/shipment); provenance is per-bar via the QR.

Pre-flight checklist (verify BEFORE writing code)

- [] **GAS deploy path for the checkout script.** Confirm which clasp mirror deploys `agroverse_shop/google-app-script/agroverse_shop_checkout/agroverse_shop_checkout.gs` (the in-repo copy is reference; deploy from `tokenomics/clasp_mirrors/<scriptId>/` per `NOTES_tokenomics.md`). Capture the `scriptId` + the existing `/exec` deployment URL so subscription sessions deploy without breaking the cart.
 - [] **Generic SKU definition.** Confirm: the canonical retail bar **price** from `agroverse_shop/js/products.js` (Elizabeth Wong batch was \$10.00/bar — verify it's still current); the **name** (proposed: `Ceremonial Cacao Chocolate Bar — Single-Estate, Rotating Origins`); a **generic GTIN** to assign (distinct from the vintage SKUs); the **hero + gallery images** (product-forward, incl. packaging-back QR shot); and the **one-off decision** (subscription-only vs. also one-off Add-to-Cart). See *Generic SKU model + PDP spec*.
 - [] **Stripe test mode.** Confirm the `sk_test` sandbox path (used for the PR6a QR-checkout work) is available to validate `mode:'subscription'` end to end before any prod promotion.
 - [] **Webhook coverage.** Confirm `sentiment_importer/app/controllers/webhook_controller.rb#stripe` currently handles only `checkout.session.completed`; Phase 2 adds `invoice.paid` (recurring renewals). Note: the **first** charge arrives as `checkout.session.completed` (mode=subscription); **renewals** arrive as `invoice.paid` — both must create a fulfillment obligation.
 - [] **Downstream session-id prefix.** The sales parser validates `cs_(live|test)_` prefixes and strips `(none)`. Subscription renewals carry `in_.../sub_...`, not `cs_...`. Decide: teach the parser the new prefix, or stash the invoice id in a separate attr and leave “Stripe Session ID” blank. (Phase 2, PR2.4.)
 - [] **Sheet writes.** Confirm the service account that may write the new “Subscription Fulfillment Queue” tab (likely `agroverse-qr-code-manager@...`); `Currencies` stays range-protected (not touched here).
 - [] **dao_client auth active** for contribution reporting (`auth.py status`).
-

Architecture (where each piece lives)

```
products.js  generic SKU entry (ONE per GTIN: generic-chocolate-bar; later ceremonial-cacao)
  subscribable:true · cadence:monthly · min/max/defaultQty · unit price · GTIN · origin:rotating
    | single source of truth — read by BOTH surfaces below
    |—————> product-page/<generic-slug>/ (PDP: one-off buy + Subscribe CTA)
    ▼
  /subscribe/chocolate-bar/ (thin clean-URL wrapper → shared subscribe engine)
    later: /subscribe/ceremonial-cacao/ (another wrapper, SAME engine, no new code)
    | engine: quantity picker + address; resolves the SKU from the catalog
    | GET ?action=createSubscriptionCheckoutSession&sku=...&qty=N&address=...
    ▼
  GAS  agroverse_shop_checkout.gs (dynamic price_data + recurring)
    | line 1: N units @ unit_amount, recurring monthly
    | line 2: shipping @ EasyPost(address), recurring monthly
    | mode:'subscription' → returns checkout URL (creates Stripe Customer)
    ▼
  Stripe Checkout (subscription) → user pays
    | checkout.session.completed (first) —┐
    | invoice.paid (every renewal)         └─┐→ Edgar webhook_controller#stripe (Rails)
    |
    ▼
  Stripe Social Media Checkout ID tab      "Subscription Fulfillment Queue" tab
  (audit, existing)                       subscriber | addr | SKU | qty | period
                                           | invoice id | status=PENDING
                                           ▼
                                           dapp_beta fulfill_subscriptions.html (Kirsten)
                                           pick obligation → scan N QR codes → tracking# → submit ONCE
                                           ▼
                                           fans out → N × [SALES EVENT] (Item=QR, Sold by=Kirsten,
                                           Cash proceeds=Gary, tag=invoice id) → status=FULFILLED
                                           ▼
                                           EXISTING chain: QR Code Sales → MINTED→SOLD →
                                           AGL ledger (per bar) → treasury cache → inventory depletes
```

Generic SKU model + PDP spec

Catalog entry (products.js) — one per generic GTIN

The generic, vintage-independent line is a normal catalog row plus subscription metadata. It is the **single source of truth** for both the PDP and the subscribe engine. Proposed shape (extends today's fields: `productId`, `productPageSlug`, `name`, `price`, `weight`, `image`, `category`):

```
{
  productId: 'generic-ceremonial-cacao-chocolate-bar',
  productPageSlug: 'ceremonial-cacao-chocolate-bar', // PDP at /product-page/<slug>/
  name: 'Ceremonial Cacao Chocolate Bar — Single-Estate, Rotating Origins',
  gtin: '<assign a generic GTIN>', // its own barcode, distinct from vintage SKUs
  price: 10.00, // unit price (confirm vs current bar price)
  weight: 1.76, // oz, per bar (50g)
  image: '/assets/images/products/generic-bar-hero.jpg',
  category: 'retail',
  origin: 'rotating', // sentinel — NOT one farm/shipment
  // subscription metadata (new):
  subscribable: true,
  subscriptionSlug: 'chocolate-bar', // → /subscribe/chocolate-bar/
  cadence: 'monthly',
  minQty: 1, maxQty: 24, defaultQty: 6, // Linda = 6/month
}
```

Adding ceremonial cacao later = **another entry** (`subscriptionSlug: 'ceremonial-cacao'`) + its wrapper + its PDP. Engine, GAS action, fulfillment queue: unchanged.

Generic-bar PDP — what goes on the page

The generic PDP is both a discoverable product page (one-off purchase) and the marketing surface that drives the subscription. It diverges from a normal single-estate PDP in one key way: **it is not bound to a single farm/shipment**, so the “Products from This Farm / Shipment” cross-listing grids in

`AGROVERSE_SHOP_NEW_SKU_WEB_CHECKLIST.md` do **not** apply. Instead it links out to “the farms we rotate through.”

Imagery (follow the PDP image convention — hero at full width, supplementary in a `.gallery` grid inside `.product-image-container`): - **Hero = the product**, not a farm portrait (there’s no single farm) — the bar + packaging front. - **Gallery**: bar close-up; **packaging back showing the traceability QR** (this is the hook — it’s how you learn the origin); optionally a montage/map of the rotating Bahia origins.

Content blocks (top → bottom): 1. **Title + the concept.** Lead copy explains the model: every bar is a single estate from a **rotating** Bahia farm; you **discover the exact farm + vintage by scanning the QR on the back**. Frame as a feature — “a monthly journey through our farms” (the Cacao Chasers surprise model) — not a limitation. 2. **Primary CTA → Subscribe.** “Subscribe — N bars/month” linking to `/subscribe/chocolate-bar/`; note **cancel/modify anytime** (Stripe Customer Portal, Phase 3). Show the default 6/month, adjustable. 3. **Secondary CTA → one-off.** Standard Add-to-Cart for a single generic bar (current stock), if we offer one-off (decision below). 4. **What you get.** 6 bars/month (configurable), shipped from SF, each QR-traceable to its estate + vintage. 5. **Provenance & traceability.** Every bar carries a QR → exact farm, vintage, on-chain ledger entry; link to the scanner / “how it works.” 6. **Tasting & spec.** House style for the rotating line (e.g. 81% dark, ceremonial-grade, minimal ingredients), ingredients, allergens, **per-bar weight (50g)**. 7. **Shipping & subscription terms.** Ships monthly; shipping computed at signup and locked; manage via the portal. 8. **Wholesale banner** between the CTA block and the description, per the SKU checklist convention (`../..//wholesale/`). 9. **Impact.** Regenerative, single-estate, supports farmers + the DAO.

Feeds / Merchant Center / JSON-LD: the **one-off** generic bar is a normal feed product (give it its own GTIN; follow `agroverse_shop/docs/PRODUCT_CREATION_CHECKLIST.md`). The **subscription** itself is a site feature, not a Merchant Center product — don’t try to model the recurring SKU in the product feed.

Open decision (one-off on the generic PDP?): offer the generic bar as a single one-off purchase too, or make it **subscription-only**? Defaulting to “one-off allowed” (the PDP doubles as a normal product page); flip to subscription-only if we don’t want to hold generic one-off stock.

Execution roadmap (resume tracker)

Legend: ☐ todo · **⚡** in progress · ☒ merged ·  DAO contribution reported

Phase 1 — Generic SKU + data-driven subscribe engine (START HERE)

Architecture: one shared engine, clean path URLs, catalog-as-source-of-truth (see *Generic SKU model + PDP spec* and the *Decisions* table). Build the engine once; expose each line via a thin wrapper.

PR	Repo	Scope	Status
1.1	<code>agroverse_shop_beta</code>	Generic SKU + schema. Add the subscription metadata fields to <code>products.js</code> (<code>subscribable</code> , <code>subscriptionSlug</code> , <code>cadence</code> , <code>min/max/defaultQty</code> , <code>gtin</code> , <code>origin: 'rotating'</code>) and the first generic-bar catalog entry (price, weight, hero image). Source of truth for all downstream.	☐

PR	Repo	Scope	Status
1.2	agroverse_shop_beta	Shared subscribe engine (<code>/subscribe/</code>): resolves the SKU from the catalog, renders quantity picker (min/max/default) + address form (reuse <code>checkout-form-storage.js</code>), calls the GAS action, redirects to Stripe. No product-specific code — fully data-driven.	☐
1.3	agroverse_shop_beta	Clean path wrapper <code>/subscribe/chocolate-bar/</code> — ~10-line page that points the engine at <code>subscriptionSlug: 'chocolate-bar'</code> . This is the placard-QR + Linda URL. (Template for future <code>/subscribe/ceremonial-cacao/</code> .)	☐
1.4	GAS (checkout mirror)	New <code>createSubscriptionCheckoutSession</code> action: unit line item with <code>recurring: {interval: 'month'}</code> ; run existing EasyPost weight+rate code once for the address → recurring shipping line item; <code>mode: 'subscription'</code> ; return <code>checkoutUrl</code> . Reuse the cart action's price/weight helpers; key off <code>sku</code> .	☐
1.5	agroverse_shop_beta	Generic-bar PDP at <code>/product-page/<generic-slug>/</code> per the <i>PDP spec</i> above: discovery/rotating-origin copy, hero=product, gallery incl. packaging-back QR shot, primary Subscribe CTA → <code>/subscribe/chocolate-bar/</code> , optional one-off Add-to-Cart, provenance block, wholesale banner. (Cross-listing grids N/A — generic is not farm/shipment-bound.)	☐
1.6	GAS / verify	Confirm subscription-mode <code>checkout.session.completed</code> still logs to the Stripe Social Media Checkout ID tab (audit) via the existing webhook path; patch if subscription sessions differ.	☐
1.7	promote + handoff	Smoke-test one subscription end-to-end in Stripe test mode ; beta → prod promotion (human); then send Linda the prod <code>/subscribe/chocolate-bar/</code> URL .	☐

Adding ceremonial cacao later = data only: one new `products.js` entry (`subscriptionSlug: 'ceremonial-cacao'`) + a `/subscribe/ceremonial-cacao/` wrapper + its PDP. The engine, GAS action, webhook, and Phase-2 fulfillment queue do **not** change.

Phase 2 — Fulfillment automation (kill the relay)

PR	Repo	Scope	Status
2.1	Google Sheets	Create “ Subscription Fulfillment Queue ” tab + schema (subscriber, email, address, SKU, qty, period, invoice id, status, fulfilled-by, tracking#, fulfilled-at).	☐
2.2	sentiment_importer (Rails)	Handle <code>invoice.paid</code> and subscription-mode <code>checkout.session.completed</code> in <code>webhook_controller#stripe</code> → append a PENDING obligation row (new <code>Gdrive::*</code> writer). Idempotent on invoice id.	☐
2.3	dapp_beta	<code>fulfill_subscriptions.html</code> : list PENDING obligations, pick one, scan/enter N QR codes (reuse <code>report_sales.html</code> + <code>edgar_payload_helper.js</code>), enter tracking#, submit once → loop N <code>[SALES EVENT]</code> s (Sold by=Kirsten, Cash proceeds=Gary, tag invoice id) → mark FULFILLED.	☐
2.4	GAS (sales parser)	Accept subscription invoice/ <code>sub_</code> ids (or read a dedicated attr) so renewal-sourced <code>[SALES EVENT]</code> s reconcile without the <code>cs_</code> guard tripping.	☐

PR	Repo	Scope	Status
2.5	dapp_beta (stretch)	Low-stock alert on the queue: warn when generic-pool stock < upcoming obligations (supply-buffer second-order effect).	☐

Phase 3 — RSA accounts + cross-browser portability

PR	Repo	Scope	Status
3.1	agroverse_shop_beta	“Sign in” in nav: reuse canonical RSA flow (publicKey/privateKey localStorage, edgar_payload_helper.js, [EMAIL REGISTERED EVENT] →link→ [EMAIL VERIFICATION EVENT]).	☐
3.2	Google Sheets / Rails	Add buyer email column to the Stripe Social Media Checkout ID tab (write it on append_record); backfill from Stripe where feasible.	☐
3.3	dao_protocol	Read-by-verified-email endpoint → returns the email’s orders (Stripe tab) + active subscriptions (Stripe API).	☐
3.4	dao_protocol	Mint a Stripe Customer Portal session for the verified email’s Customer; “Manage subscription” button on the shop.	☐
3.5	agroverse_shop_beta	/order-history/ reads the server endpoint when signed in (localStorage = fallback cache); one-time migration of local agroverse_order_history on first verify.	☐

Risks / open items

- **Supply buffer.** Subscriptions create a recurring inventory commitment; the generic pool must be replenished as vintages finish. PR2.5 surfaces it; supply planning is out of scope for code but in scope for Dr-Manhattan-level attention.
- **Shipping drift.** Locked-at-signup shipping can diverge from real cost over time (carrier increases, address-region changes). Acceptable for v1; revisit if margins slip. Customer Portal lets subscribers update address — decide whether an address change should re-quote shipping (likely a manual re-quote at first).
- **Deferred revenue timing.** Charge lands on billing day; revenue recognized when QRs are scanned days later. The obligation queue *is* the deferred-revenue tracker — correct, but means the Stripe tab and per-bar [SALES EVENT]s are temporarily out of sync within a cycle (expected).
- **Single-vs-multi service shipping.** EasyPost returns multiple USPS rates; v1 picks one tier (e.g. Ground Advantage) computed for the address rather than letting the subscriber choose. Confirm the tier in PR1.2.

Plan owner: this doc. Update the resume tracker as each PR lands; report the DAO contribution before starting the next PR.